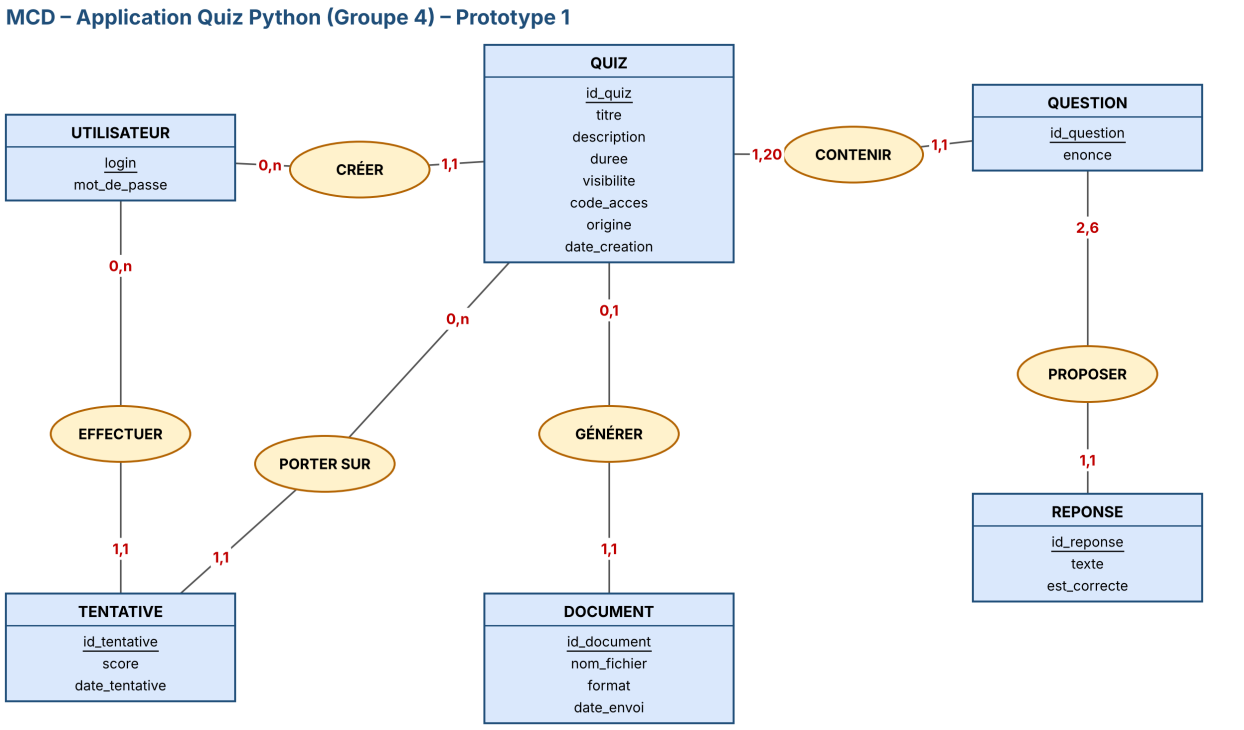
Modèle conceptuel de données (MCD) – Prototype 1

PROJ531 – Groupe 4 – Application Quiz Python · 23/10/2026



Identifiants soulignés. Cardinalités en rouge, écrites du côté de l'entité concernée.

1. Entités

Entité	Identifiant	Attributs	Rôle
UTILISATEUR	login	mot_de_passe	Compte : chaque utilisateur peut créer des quiz et jouer.
QUIZ	id_quiz	titre, description, duree, visibilite, code_acces, origine, date_creation	duree en secondes ; visibilite = public ou prive ; code_acces vide si public ; origine = saisie, import ou generation.
QUESTION	id_question	enonce	Question d'un quiz.
REPONSE	id_reponse	texte, est_correcte	Réponse proposée pour une question.
TENTATIVE	id_tentative	score, date_tentative	Un passage de quiz : sert à l'historique.
DOCUMENT	id_document	nom_fichier, format, date_envoi	Document source d'un quiz généré (format = txt ou pdf).

2. Associations et cardinalités

Association	Cardinalités	Lecture
CRÉER	UTILISATEUR 0,n – QUIZ 1,1	Un utilisateur crée 0 ou plusieurs quiz ; un quiz a un seul créateur (son administrateur).
CONTENIR	QUIZ 1,20 – QUESTION 1,1	Un quiz contient de 1 à 20 questions ; une question appartient à un seul quiz.
PROPOSER	QUESTION 2,6 – REPONSE 1,1	Une question propose de 2 à 6 réponses ; une réponse appartient à une seule question.
EFFECTUER	UTILISATEUR 0,n – TENTATIVE 1,1	Un utilisateur fait 0 ou plusieurs tentatives ; une tentative est faite par un seul utilisateur.
PORTER SUR	QUIZ 0,n – TENTATIVE 1,1	Un quiz est tenté 0 ou plusieurs fois ; une tentative porte sur un seul quiz.
GÉNÉRER	DOCUMENT 1,1 – QUIZ 0,1	Un document génère un seul quiz ; un quiz provient de 0 ou 1 document.

3. Règles de gestion

- Tout utilisateur connecté peut créer un quiz (saisie, import ou génération) ; il en devient l'administrateur.
- Seul le créateur peut modifier ou supprimer son quiz ; un joueur ne peut jamais le modifier.
- Un quiz public apparaît dans la liste ; un quiz privé s'ouvre avec son code d'accès unique de 6 caractères.
- Un quiz contient de 1 à 20 questions ; chaque question a de 2 à 6 réponses, dont exactement une correcte.
- Un quiz généré peut être relu et modifié avant d'être enregistré ; son contenu est ensuite géré comme un quiz normal.
- Le contenu du document n'est pas stocké : il est envoyé au service d'IA (Groq) puis oublié.
- Chaque passage de quiz crée une nouvelle tentative : les anciens scores sont conservés (historique).

4. MLD (passage aux tables)

Clé primaire soulignée, clé étrangère précédée de #. Une association 1,1 devient une clé étrangère du côté 1,1.

- **UTILISATEUR** (login, mot_de_passe)
- **QUIZ** (id_quiz, titre, description, duree, visibilite, code_acces, origine, date_creation, #login_createur)
- **QUESTION** (id_question, enonce, #id_quiz)
- **REPONSE** (id_reponse, texte, est_correcte, #id_question)
- **TENTATIVE** (id_tentative, score, date_tentative, #login, #id_quiz)
- **DOCUMENT** (id_document, nom_fichier, format, date_envoi, #id_quiz)

5. Écarts avec le schéma proposé par Olivier (23/10, 17h41)

Table	À faire
attempts	Corriger « IF NOT EXIST » en « IF NOT EXISTS » (sinon l'application plante) ; ajouter id_quizz
quizz	Ajouter origine ; supprimer subject, difficulty et link (ou retirer NOT NULL) : ni le formulaire ni l'import ne les remplissent
utilisateurs	OK (score et role retirés) ; mettre password en NOT NULL ; adapter les INSERT et supprimer l'ALTER TABLE qui rajoute role
Toutes	Déclarer les clés étrangères (REFERENCES) et activer PRAGMA foreign_keys = ON
documents	Nouvelle table à créer